

INDEX

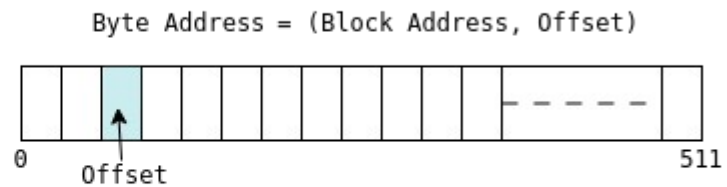
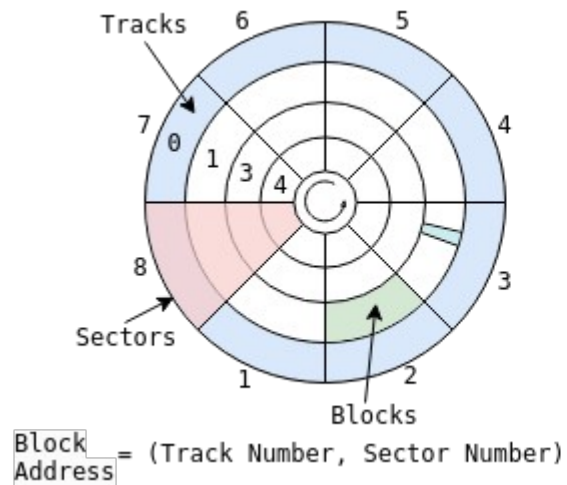
Giới thiệu

- ❖ Chỉ mục (index) là cách tốt nhất để tăng hiệu năng truy vấn, thậm chí còn quan trọng hơn cả thiết kế schema. Mỗi chỉ mục là một cấu trúc dữ liệu riêng, lưu bản sao một phần dữ liệu từ bảng
- ❖ Index là 1 cấu trúc dữ liệu riêng biệt và lưu trữ một phần dữ liệu của bảng chính
- ❖ Tạo quá nhiều Index cũng sẽ làm ảnh hưởng đến hiệu năng của hệ thống
- ❖ Query Pattern $\Leftrightarrow$ Indexing

Index

- ❖ KEY | POINTER
- ❖ Sorted hoặc Hash

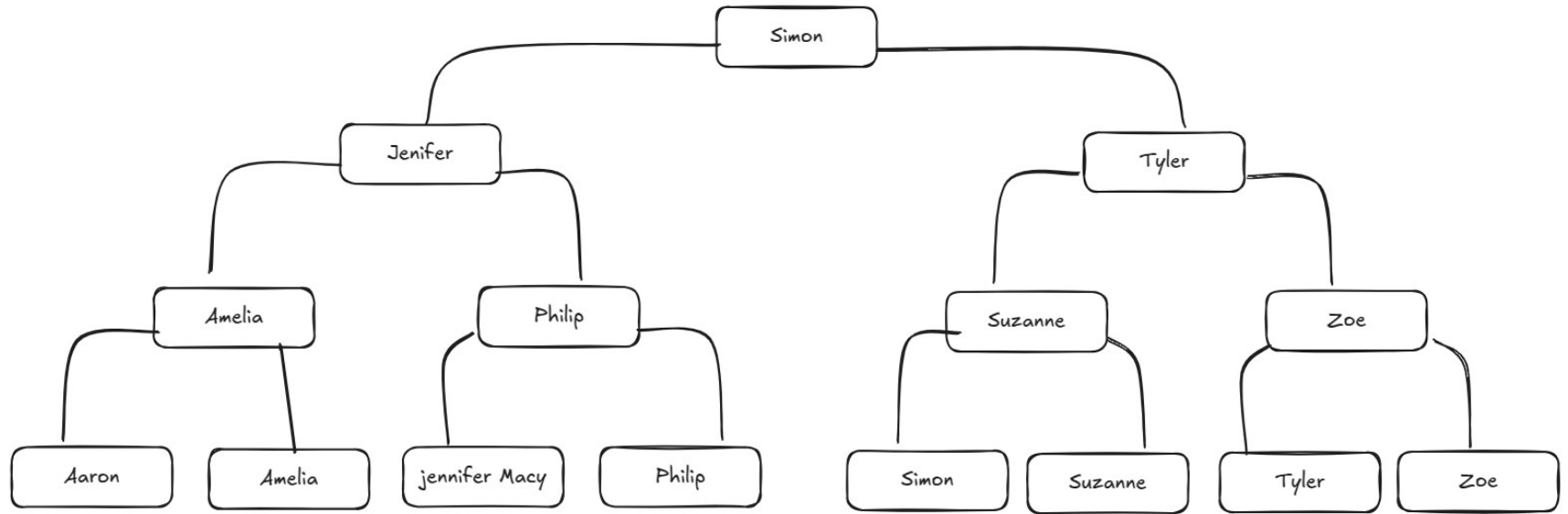
Cấu trúc Disk



Chuẩn bị

```
CREATE TABLE IF NOT EXISTS users (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL  
);  
  
-- Chèn dữ liệu từ hình ảnh  
INSERT INTO users (id, name, email) VALUES  
(1, 'Phillip', 'Phillip@example.com'),  
(2, 'Aaron', 'Aaron@example.com'),  
(3, 'Amelia', 'Amelia@example.com'),  
(4, 'Jennifer', 'Jennifer@example.com'),  
(5, 'Macy', 'Macy@example.com'),  
(6, 'Simon', 'Simon@example.com'),  
(7, 'Tyler', 'Tyler@example.com'),  
(8, 'Suzanne', 'Suzanne@example.com'),  
(9, 'Zoe', 'Zoe@example.com');
```

B+ tree



B+ tree

❖ Đặc điểm

- Tất cả dữ liệu đều nằm ở nút lá
- Nút trung gian chỉ chứa khóa định hướng
- Các nút lá được liên kết với nhau như một danh sách liên kết, cho phép duyệt tuần tự hiệu quả
- Các node lá có cùng độ cao

B+ tree

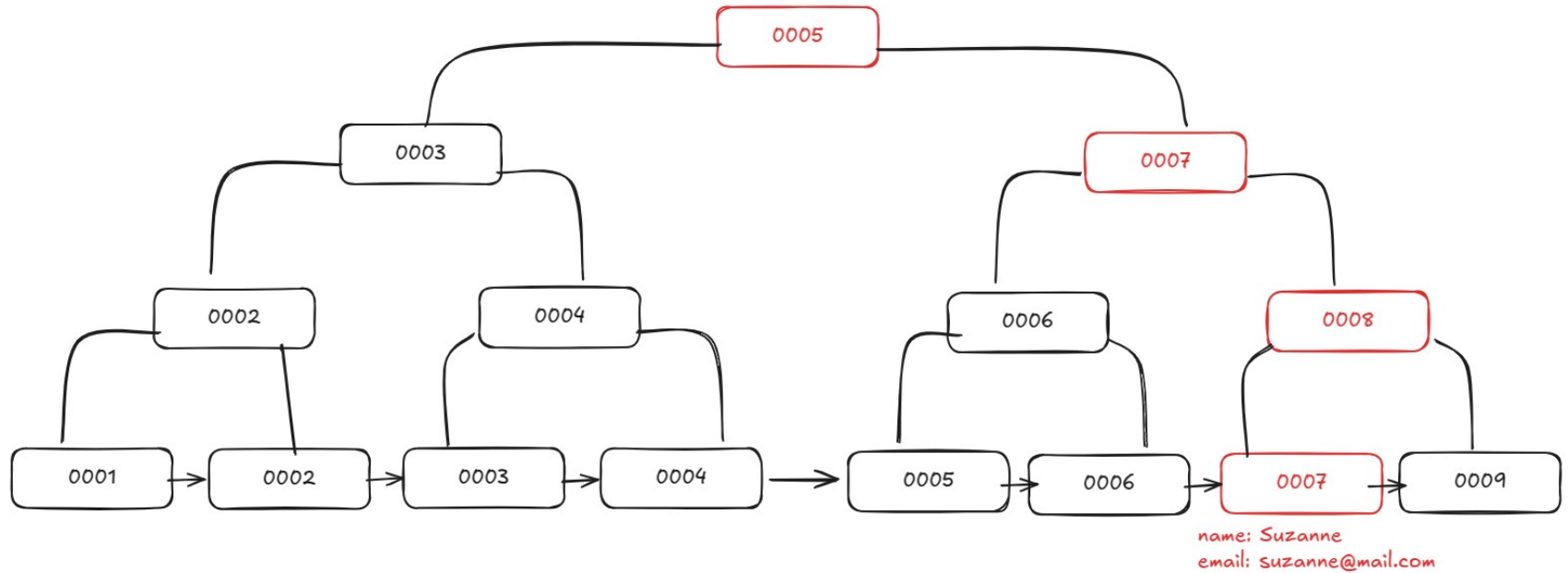
- ❖ Ưu điểm
 - Cho phép tìm kiếm với độ phức tạp $O(\log n)$, rất hiệu quả với dữ liệu lớn
 - Thiết kế để giảm thiểu số lần truy cập đĩa khi tìm kiếm, đọc/ghi dữ liệu
 - Tự động cân bằng cây khi thực hiện thao tác thêm, xóa, sửa
- ❖ Trong MySQL (như InnoDB) và nhiều hệ CSDL khác, B+ tree là cấu trúc chính được sử dụng cho các chỉ mục để tăng tốc truy vấn và tối ưu hóa hiệu suất.

Clustered VS Non-Clustered

Clustered Index

- ❖ Là index xác định thứ tự vật lý mà dữ liệu được lưu trữ trong bảng
- ❖ Đặc điểm
 - Mỗi bảng chỉ có tối đa 1 clustered index
 - Dữ liệu của bảng được **sắp xếp theo thứ tự của clustered index**
 - Truy xuất dữ liệu nhanh hơn vì dữ liệu được lưu trữ cùng với index
 - Trong MySQL/InnoDB, clustered index mặc định được tạo trên **PRIMARY KEY**

Clustered Index



Non-Clustered Index (Secondary Index)

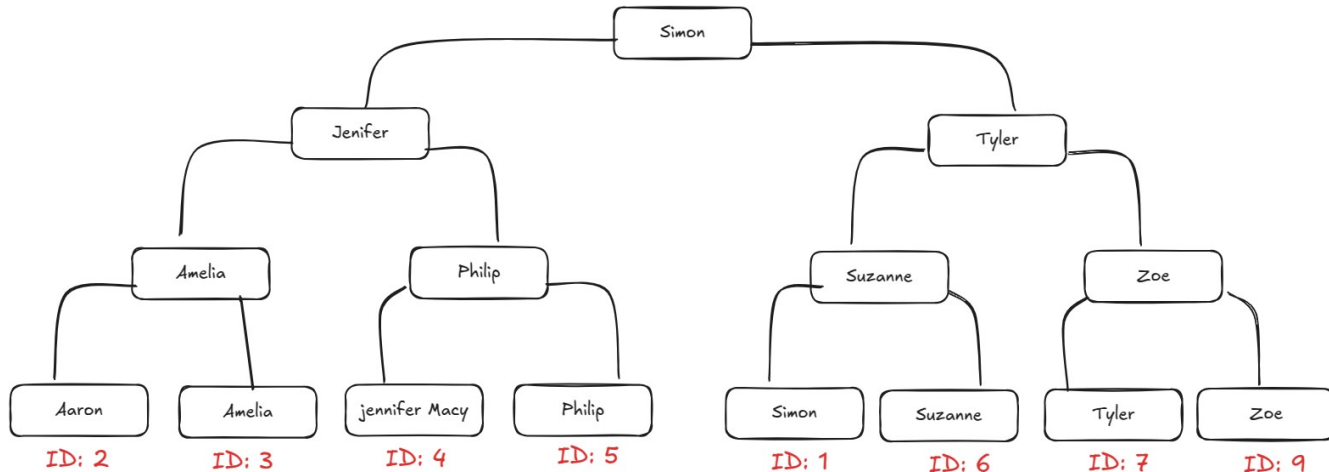
- ❖ Là index lưu trữ cấu trúc dữ liệu riêng biệt so với dữ liệu thực
- ❖ Đặc điểm
 - Một bảng có thể có **nhiều** non-clustered index
 - Index chứa **khóa và pointer** thường trỏ đến khóa của Clustered Index
 - Chậm hơn clustered index vì **cần thêm một bước tra cứu để đọc dữ liệu**
 - Việc thêm/sửa dữ liệu nhanh hơn vì không cần sắp xếp lại toàn bộ dữ liệu
 - Có thể cho phép trùng lặp và NULL

Non-Clustered Index (Secondary Index)

- ❖ Khi bảng đã có một Clustered Index (thường là Primary Key), Secondary Index sẽ lưu trữ giá trị khóa của Clustered Index như một pointer, không phải địa chỉ vật lý trực tiếp của bản ghi
- ❖ Quá trình truy vấn gồm 2 bước:
 - Tìm kiếm trong Secondary Index để lấy giá trị khóa Clustered
 - Sử dụng giá trị này để tìm kiếm trong Clustered Index và truy cập dữ liệu
 - "Adams" -> employee_id = 101
 - "Brown" -> employee_id = 205
 - "Smith" -> employee_id = 156

Non-Clustered Index (Secondary Index)

- ❖ Chọn primary key nhỏ gọn (INT, BIGINT) để tối ưu kích thước secondary index
- ❖ Nên sử dụng primary key (và do đó clustered index) cho các bảng InnoDB



Ví dụ

-- Tạo bảng với primary key (tự động trở thành clustered index)

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY, -- Clustered index  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    email VARCHAR(100),  
    hire_date DATE  
);
```

-- Thêm các secondary key (non-clustered index)

```
CREATE INDEX idx_last_name ON employees(last_name); -- Non-clustered index  
CREATE UNIQUE INDEX idx_email ON employees(email); -- Non-clustered index
```

So sánh

Đặc điểm	Clustered Index	Non-Clustered Index
Tốc độ đọc	Nhanh hơn	Chậm hơn
Tốc độ sửa	Chậm hơn	Nhanh hơn
Số lượng / bảng	Chỉ 1	Nhiều

Unique VS Non-Unique

Unique Index VS Non-Unique Index

❖ Unique Index:

- Không cho phép trùng lặp giá trị
- Thường dùng cho email, username, số điện thoại

❖ Non-Unique Index:

- Cho phép trùng lặp giá trị
- Dùng cho các cột thường xuyên xuất hiện trong điều kiện tìm kiếm

- ❖ Ví dụ: Trong hệ thống, unique index trên email đảm bảo không có hai tài khoản cùng email, trong khi non-unique index trên status hoặc mã khách hàng giúp tìm đơn hàng tốt hơn.

Khi nào nên sử dụng Index

- ❖ It means that you should start by analyzing the access patterns of your application before deciding where to put indexes.
- ❖ Before adding an index, ask yourself, which queries are you running frequently and which tables are they accessing? By analyzing the access patterns of your queries, you can better determine which indexes will be most useful. Keep in mind that not all queries require indexes, and adding too many indexes can actually harm your database's performance.

Khi nào nên sử dụng Index

- ❖ Do not assume that anything that shows up in the where clause of a query should have an index. Consider all queries being run and their respective access patterns.
- ❖ Do not create an index on every column. This will slow down inserts by functionally duplicating your table. It also won't help reads as much as you'd hope.
- ❖ Do not worry about trying to create the perfect index for every query. It may not always be possible, and sometimes you will have to rework the queries to take advantage of existing indexes.

Khi nào nên sử dụng Index

- ❖ Cho các cột thường xuyên xuất hiện trong mệnh đề :
 - WHERE
 - JOIN
 - ORDER BY
 - GROUP BY
- ❖ Cho các cột có tính chọn lọc cao (email, unique column, ...)
- ❖ So sánh bằng, range

Querying và Indexing

- ❖ Querying và Indexing là đường 2 chiều
 - Query pattern phải thay đổi để tận dụng được index
 - Index thay đổi khi các query phổ biến chuyển sang sử dụng 1 pattern khác

Index Cardinality and Selectivity

- ❖ Index Selectivity (tính chọn lọc của index) là thước đo mức độ độc đáo của các giá trị trong một cột được đánh index.
- ❖ **$\text{Selectivity} = (\text{Số giá trị duy nhất} / \text{Tổng số bản ghi}) \times 100\%$**
- ❖ Ví dụ:
 - Cột id: 1000 giá trị khác nhau trong bảng 1000 bản ghi $\rightarrow 100\%$
 - Cột gender: ~ 2 giá trị khác nhau trong bảng 1000 bản ghi $\rightarrow 0.2\%$

Tại sao Index Selectivity nhỏ thì không tốt?

- ❖ Khi database cần truy xuất dữ liệu, query optimizer đứng trước hai lựa chọn:
 - **Table Scan**: Quét toàn bộ bảng để tìm dữ liệu phù hợp
 - **Index Seek + Table Lookup**: Tìm trong index, sau đó truy cập bảng gốc
- ❖ Nhắc lại: Quá trình khi sử dụng index:
 - Tìm kiếm trong cấu trúc index (thường là B-tree) $\rightarrow O(\log n)$
 - Với mỗi khóa phù hợp, truy cập bảng gốc bằng **random access**

Tại sao Index Selectivity nhỏ thì không tốt?

- ❖ Query Optimizer của DBMS ước tính chi phí của cả hai phương án và chọn phương án tối ưu. Nó cân nhắc:
 - Chi phí truy cập index
 - Chi phí truy cập bảng gốc (lookup)
 - Số lượng bản ghi cần truy xuất
- ❖ Ngưỡng quyết định quan trọng:
 - Nếu truy vấn trả về một tỷ lệ **nhỏ** bản ghi → Sử dụng **Index** hiệu quả
 - Nếu truy vấn trả về một tỷ lệ **lớn** bản ghi → **Table scan** hiệu quả hơn

Sequential Access VS Random Access

❖ Khi sử dụng index:

- Tìm trong index và nhận được ~500 khóa (với 1000 bản ghi)
- Thực hiện ~500 lần lookup ngẫu nhiên vào bảng gốc
- Tổng chi phí: 1 lần tìm kiếm index + 500 lần random access

❖ Khi sử dụng table scan:

- Đọc tuần tự toàn bộ 1000 bản ghi
- Tổng chi phí: 1000 lần **sequential access**
- nhanh hơn nhiều so với **random access**

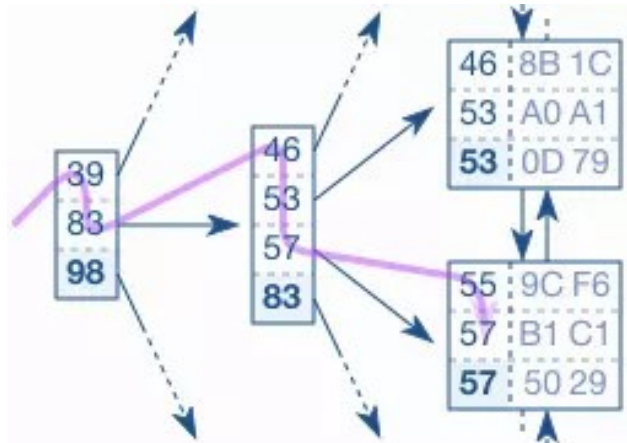
Thử
xem



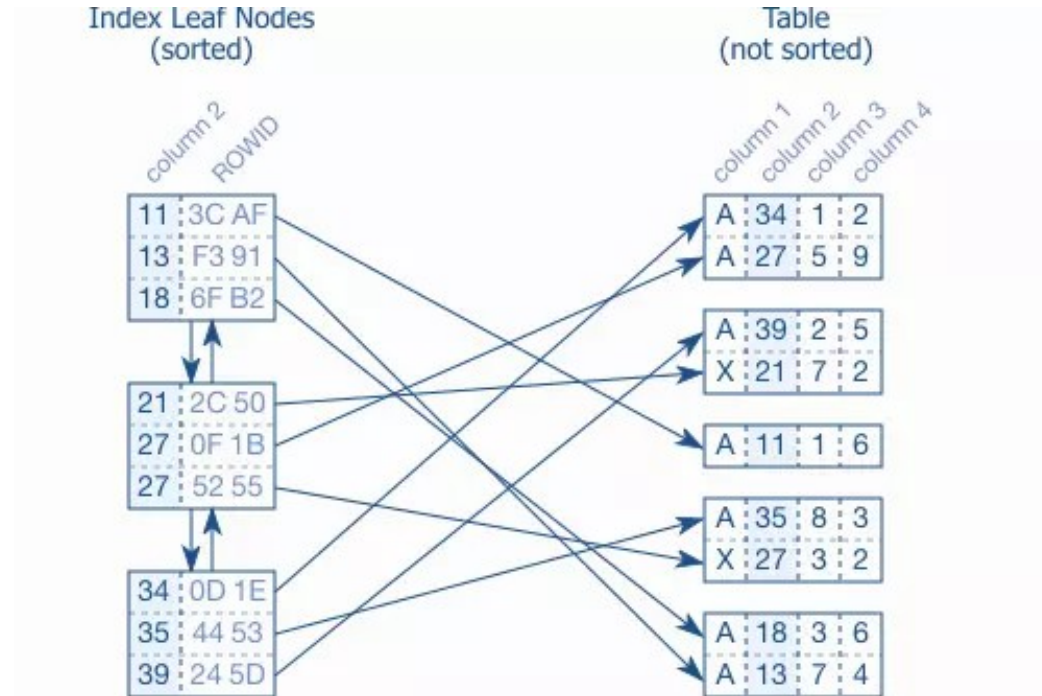
Chiến lược cho các Index Selectivity nhỏ

- ❖ Full Table Scan không phải lúc nào cũng tệ. Trong trường hợp lấy dữ liệu lớn từ bảng, Full Table Scan có thể đọc nhiều block cùng một lúc (multi block read) nên dù phải đọc nhiều dữ liệu hơn nhưng nó tốn ít thao tác đọc hơn
- ❖ Kết hợp trong Composite Index
- ❖ Sử dụng Filtered Index (Partial index)
 - Sẽ trình bày sau

Một số nguyên nhân khiến Index chậm



Bước duyệt theo leaf node chậm



leaf node chứa nhiều cục index

Một số nguyên nhân khiến Index chậm

- ❖ So sánh chiến lược thực thi
 - INDEX UNIQUE SCAN
 - INDEX RANGE SCAN
 - Truy vấn trên khóa chính (Clustered Index)

PREFIX INDEX

Prefix Index

- ❖ Prefix Index là một loại index đặc biệt mà **chỉ lưu trữ một phần đầu** (tiền tố) của cột dữ liệu thay vì toàn bộ nội dung. Đây là cách tối ưu rất hiệu quả khi làm việc với các cột có kiểu **dữ liệu dài** như VARCHAR (>100), TEXT, BLOB, nhất là khi các giá trị có phần đầu **đủ khác biệt**.
- ❖ "[john.smith@example.com](#)" ⇒ Prefix Index (10): **"john.smith"**
- ❖ Khi truy vấn, DBMS sẽ:
 - Tìm kiếm các bản ghi có tiền tố phù hợp trong index
 - Truy cập bảng gốc để xác minh dữ liệu đầy đủ (nếu cần)

```
CREATE INDEX idx_name ON table_name(column_name(N));
```

Prefix Index

❖ Ưu điểm

- Giảm kích thước Index đáng kể ⇒ Nhiều khóa hơn trên mỗi trang index
- Chi phí cập nhật index khi thêm/sửa/xóa dữ liệu cũng giảm
- Tăng hiệu suất I/O và Cache
- Hiệu quả cho tìm kiếm với tiền tố. Ví dụ: WHERE email LIKE 'john.s%';

❖ Nhược điểm

- Giảm tính chọn lọc (selectivity)
- Prefix index không phù hợp cho **order by, group by** vì Index không chứa toàn bộ giá trị chuỗi

Nguyên tắc chọn Prefix Index

❖ Nguyên tắc chọn độ dài

- Chọn độ dài sao cho tính chọn lọc đạt ít nhất **80-90%** so với index đầy đủ
- Cân bằng kích thước và hiệu suất: Bắt đầu từ 5-10 ký tự, tăng dần cho đến khi đạt tính chọn lọc cần thiết

```
SELECT COUNT(DISTINCT LEFT(first_name, 4)) / COUNT(*) as left4 from people;  
SELECT COUNT(DISTINCT LEFT(first_name, 5)) / COUNT(*) as left5 from people;  
SELECT COUNT(DISTINCT LEFT(first_name, 6)) / COUNT(*) as left6 from people;  
SELECT COUNT(DISTINCT LEFT(first_name, 7)) / COUNT(*) as left7 from people;
```

Composite Indexes

Composite Index

- ❖ Composite Index (hay Multi-column Index) là index được tạo trên **nhều cột kết hợp**, thay vì chỉ một cột duy nhất. Composite index giúp tối ưu hóa hiệu suất truy vấn khi điều kiện tìm kiếm hoặc sắp xếp liên quan đến **nhều cột** cùng lúc.
- ❖ Về mặt kỹ thuật, Composite index tạo ra một cấu trúc B-tree với **mỗi khóa là sự kết hợp các giá trị từ nhiều cột**. Các giá trị này được sắp xếp theo **thứ tự khai báo trong index**

```
CREATE INDEX idx_name ON table_name(column1, column2, column3);  
CREATE INDEX idx_customer_order ON orders(customer_id, order_date, status);
```

Composite Index VS Single-column Index

Đặc điểm	Single-column Index	Composite Index
Số cột	1	2+
Kích thước	Nhỏ hơn	Lớn hơn
Tính đa năng	Ít	Nhiều
Độ phức tạp	Đơn giản	Phức tạp hơn
Tối ưu cho	Điều kiện một cột	Nhiều điều kiện kết hợp

Composite Index

- ❖ Composite Index (hay Multi-column Index) là index được tạo trên **nhều cột kết hợp**, thay vì chỉ một cột duy nhất. Composite index giúp tối ưu hóa hiệu suất truy vấn khi điều kiện tìm kiếm hoặc sắp xếp liên quan đến **nhều cột** cùng lúc.
- ❖ Về mặt kỹ thuật, Composite index tạo ra một cấu trúc B-tree với **mỗi khóa là sự kết hợp các giá trị từ nhiều cột**.
- ❖ Các giá trị này được sắp xếp theo **Thứ tự khai báo trong index**

```
CREATE INDEX idx_name ON table_name(column1, column2, column3);  
CREATE INDEX idx_customer_order ON orders(customer_id, order_date, status);
```

Composite Index

❖ Quy tắc

- Tìm từ **trái sang phải** theo thứ tự khai báo, không bỏ qua (leftmost prefix)
- Dừng lại khi gặp **Range đầu tiên**
- ❖ key_len trong khi Explain là số lượng byte sử dụng cho index trong query
- ❖ Tạo 1 index: **multi**(first_name, last_name, birthday)
 - Query firstName
 - Query firstName + lastName
 - Query firstName + lastName + birthday

Nguyên tắc xác định thứ tự trong Composite Index

- ❖ Nguyên tắc theo tính chọn lọc (Selectivity)
 - Đặt cột có tính chọn lọc cao nhất ở vị trí bên trái nhất.
- ❖ Ưu tiên thứ tự cột dựa trên mẫu truy vấn (Query Pattern) thường gặp nhất.
 - Trên tầng ứng dụng, nên thống nhất 1 kiểu filter, search
 - Tùy thuộc vào tần suất sử dụng mỗi truy vấn (70%, 20%, 10%)
- ❖ Các cột thường truy vấn dạng RANGE thì sẽ đẩy về cuối (VD: order_date)

Khi nào nên sử dụng Composite Index

- ❖ Truy vấn với nhiều điều kiện WHERE kết hợp

```
SELECT * FROM products WHERE category_id = 5 AND brand_id = 10 AND price > 100;
```

- ❖ Truy vấn có ORDER BY kết hợp với WHERE

```
SELECT * FROM customers  
WHERE status = 'active' AND country = 'USA'  
ORDER BY last_name, first_name;
```

```
CREATE INDEX idx_status_country_name ON customers(status, country, last_name,  
first_name);
```


Khi nào nên sử dụng Composite Index

- ❖ Truy vấn có GROUP BY kết hợp với WHERE

```
SELECT position, COUNT(*)  
FROM employees  
WHERE departmentId = 5  
GROUP BY position;  
  
-- Composite index hiệu quả  
CREATE INDEX idx_d ON employees(departmentId, position);
```

- ❖ Thay thế nhiều Single-column Index
 - Thay thế hiệu quả các single-column index khi tuân thủ quy tắc leftmost prefix. Và phần lớn các truy vấn phức tạp liên quan đến nhiều cột

Use case

- ❖ Tìm đơn hàng theo khách hàng + khoảng thời gian

```
SELECT * FROM orders WHERE customer_id = ? AND order_date BETWEEN ? AND ?;  
CREATE INDEX idx_customer_date ON orders(customer_id, order_date);
```

- ❖ Tìm đơn hàng theo khách hàng + trạng thái

```
SELECT * FROM orders WHERE customer_id = ? AND status = ?;  
CREATE INDEX idx_customer_status ON orders(customer_id, status);
```

Bài tập

- ❖ Thường xuyên tìm kiếm theo mã phòng ban
- ❖ Thường xuyên tìm kiếm theo mã phòng ban, position (PM, BA, Dev, Lead, ...)

Covering Index

Covering Index

- ❖ Quan sát phần EXTRA trong EXPLAIN $\Rightarrow$ “Using Index”
- ❖ Nhắc lại về:
 - clustered index (primary key)
 - non-clustered index (secondary index)
- ❖ Covering Index là 1 Index thông thường nhưng trong 1 hoàn cảnh đặc biệt

Covering Index

- ❖ Covering Index (chỉ mục bao phủ) trong MySQL là một index được coi là "covering" cho một truy vấn khi **tất cả các cột xuất hiện trong truy vấn đều nằm trong index**. Với covering index, MySQL có thể trả về kết quả trực tiếp từ index mà không cần truy cập vào bảng gốc, giảm đáng kể chi phí I/O.
 - Tất cả dữ liệu cần thiết cho câu truy vấn đều nằm trong index
 - Không cần "table lookup" (truy cập vào bảng gốc)
 - Nhận biết bằng "Using index" trong EXPLAIN

Covering Index

❖ Covering Index với khóa chính

```
-- Với PK là id
CREATE INDEX idx_dept ON employees (name);

-- Truy vấn này có thể sử dụng covering index
SELECT id, name
FROM employees
WHERE name = 'Lisa';
```

Covering Index

❖ Covering Index cơ bản

```
SELECT first_name, last_name  
FROM employees  
WHERE department_id = 5;
```

```
CREATE INDEX idx_dept_name ON employees(department_id, first_name, last_name);
```


Covering Index

❖ Covering Index với order by

-- Tạo index bao gồm cả cột sắp xếp

```
CREATE INDEX idx_dept_salary ON employees(department_id, salary, first_name, last_name);
```

-- Truy vấn sử dụng Covering Index

```
SELECT first_name, last_name, salary
```

```
FROM employees
```

```
WHERE department_id = 5
```

```
ORDER BY salary DESC;
```

Covering Index - Ví dụ

```
-- 1,4s
```

```
select SQL_NO_CACHE id, user_id, status, order_date  
from orders IGNORE INDEX (idx_user_status_date)  
where user_id = 108 and status != 'Đang xử lý' and order_date > '2022-01-01'
```

```
-- 0.17 s
```

```
select SQL_NO_CACHE id, user_id, status, order_date  
from orders FORCE INDEX (idx_user_status_date)  
where user_id = 108 and status != 'Đang xử lý' and order_date > '2022-01-01'
```

Covering Index

- ❖ Thiết kế covering index cần được cân nhắc kỹ lưỡng, đánh đổi giữa hiệu suất truy vấn và chi phí lưu trữ/bảo trì.
- ❖ “Truy vấn nhiều, cập nhật ít”
- ❖ Cân nhắc:
 - Sự phân bố dữ liệu của bảng
 - Không phải nhiều trường là lâu, ít trường là nhanh
 - Hiểu cả nghiệp vụ lẫn kỹ thuật ⇒ Tối ưu
- ❖ <https://data-mozart.com/indexing-for-better-performance-real-use-case/>

Functional indexes

Functional indexes

MySQL hỗ trợ functional index từ phiên bản 8.0

```
-- Tìm người dùng có ngày sinh trong tháng cụ thể  
SELECT * FROM users WHERE MONTH(birthday) = 6;
```

```
-- Index cho tìm kiếm theo tháng  
CREATE INDEX idx_reg_month ON users(MONTH(birthday));
```

```
- MySQL  
ALTER TABLE users  
  ADD COLUMN birthday_month VARCHAR(255) AS (MONTH(birthday));  
CREATE INDEX emp_up_name ON users (birthday_month);
```

Functional indexes

- ❖ **Functional Index** (Index dựa trên hàm) là một loại index được tạo trên **kết quả của một biểu thức hoặc hàm** được áp dụng lên một hay nhiều cột, **thay vì trực tiếp trên giá trị gốc của cột**.
- ❖ Functional indexes cho phép tối ưu hóa các truy vấn có điều kiện tìm kiếm áp dụng cùng **hàm hoặc biểu thức** trên cột dữ liệu.
- ❖ Vì khi cho cột vào trong function $\Rightarrow$ Xảy ra hiện tượng *obfuscated* / *blackbox*

Functional indexes

Với trường mã sinh viên theo format cố định 'SV_xxx' (tiền tố 'SV_' và phần hậu tố biến đổi), ví dụ: SV_001, SV_002

```
-- Thường xuyên truy vấn theo dạng
```

```
SELECT * FROM students WHERE SUBSTRING(student_id, 4) > '500';
```

```
-- Sử dụng Functional Index
```

```
CREATE INDEX idx_student_number ON users((SUBSTRING(student_id, 4)));
```

Functional indexes - Lưu ý

- ❖ Ưu tiên index trên dữ liệu gốc thay vì function-base nếu không thực sự cần thiết
- ❖ Hạn chế mỗi người trong team 1 phong cách code khác nhau cho cùng 1 bài toán

Index for wildcard

Index for wildcard

- ❖ Index truyền thống (B-tree) chỉ có thể sử dụng hiệu quả cho đến khi gặp ký tự **wildcard đầu tiên**

```
-- Thêm vào bảng hiện có
```

```
ALTER TABLE users ADD INDEX idx_name (name);
```

```
-- Truy vấn
```

```
SELECT * FROM users WHERE name LIKE "Jo%n"
```

```
SELECT * FROM users WHERE name LIKE "%Smith"
```

FULLTEXT INDEX

Fulltext Index

- ❖ FULLTEXT INDEX (chỉ mục toàn văn) là một loại chỉ mục đặc biệt được thiết kế để **tối ưu hóa tìm kiếm văn bản** trong các cột chứa nội dung dạng text. *Khác với các loại index thông thường (B-tree, Hash), Fulltext index không chỉ so khớp chính xác mà còn hỗ trợ tìm kiếm ngữ nghĩa, xếp hạng độ liên quan, và nhiều tính năng tìm kiếm văn bản nâng cao.*

Fulltext Index

-- Thêm vào bảng hiện có

ALTER TABLE articles

ADD FULLTEXT INDEX ft_title_content (title, content);

-- Tạo riêng

CREATE FULLTEXT INDEX ft_content ON articles(content);

Fulltext Index

```
SELECT id, first_name, bio,  
       MATCH(bio) AGAINST('Something last rock' IN NATURAL LANGUAGE MODE) rate  
FROM users  
WHERE MATCH(bio) AGAINST('Something last rock' IN NATURAL LANGUAGE MODE)  
LIMIT 50;
```

Fulltext Index

```
SELECT id, first_name, bio
FROM users
WHERE MATCH(bio) AGAINST('+Something +couple +environment java -python' IN BOOLEAN MODE)
LIMIT 50;
```

Fulltext Index

```
-- Tìm kiếm cơ bản (Natural Language Mode)
```

```
SELECT * FROM articles
```

```
WHERE MATCH(title, content) AGAINST('database optimization');
```

```
-- Tìm kiếm Boolean Mode
```

```
SELECT * FROM articles
```

```
WHERE MATCH(content) AGAINST('database +performance -cloud' IN BOOLEAN MODE);
```


Fulltext Index **Natural Language Mode**

❖ **Natural Language Mode (mặc định)**

- Tìm kiếm tự nhiên, xếp hạng theo mức độ liên quan
- Kết quả phải đạt ngưỡng liên quan tối thiểu
- `MATCH(content) AGAINST('database optimization')`

Fulltext Index **Boolean Mode**

❖ Boolean Mode

- Sử dụng toán tử Boolean: +, -, >, <, (), *
- Không có ngưỡng liên quan tối thiểu
- `MATCH(content) AGAINST('+database +performance -cloud "abc"' IN BOOLEAN MODE)`
 - +term: Bắt buộc có term
 - -term: Không được có term
 - term*: Tìm tiền tố (prefix matching)
 - "exact phrase": Tìm cụm từ chính xác

Fulltext Index - Ví dụ tối ưu

```
SELECT id, first_name, bio
FROM users
WHERE MATCH(bio) AGAINST('*Something*' IN BOOLEAN MODE)
```

```
select id, first_name, bio
from users
where bio LIKE '%Something%'
```

Partial Index (Filtered Index)

Partial Index

- ❖ **Partial Index (hay Filtered Index)** chỉ đánh index cho một **tập con** dữ liệu trong bảng, dựa trên một **điều kiện được xác định**. Thay vì đánh index cho tất cả các hàng, nó chỉ đánh index những hàng đáp ứng điều kiện WHERE cụ thể
 - Nhỏ hơn so với index thông thường vì chứa ít dữ liệu hơn
 - Được hỗ trợ bởi PostgreSQL, SQL Server (gọi là Filtered Index), SQLite, và một số DBMS khác, không có trong MySQL =(
 - Tối ưu hóa truy vấn cho các tập dữ liệu con thường xuyên được truy cập
 - Rất hữu ích khi bảng có phân phối dữ liệu không đều hoặc khi các truy vấn thường tập trung vào một phần cụ thể của dữ liệu

Partial Index

```
CREATE INDEX idx_status_order ON orders (ship_address)
WHERE order_status = 'Đang giao';
```

“Duplicate” Index

“Duplicate” Index

```
ALTER TABLE people ADD INDEX first_name (first_name);  
ALTER TABLE people ADD INDEX full_name (first_name, last_name, birthday);  
SELECT * FROM people WHERE first_name = 'Aaron' ORDER BY id DESC;
```


Hash Index

Hash Index

- ❖ **Hash Index** là một Index sử dụng hàm băm (hash function) để ánh xạ các giá trị cột thành các giá trị băm, sau đó được sử dụng để định vị dữ liệu nhanh chóng.
 - Cung cấp thời gian tìm kiếm nhanh cho các **truy vấn khớp chính xác**
 - Kích thước thường nhỏ hơn các chỉ mục B-tree
 - Hash có tốc độ nhanh hơn kiểu Btree
 - Chỉ hỗ trợ so sánh bằng (=)
 - **Không** hỗ trợ các truy vấn RANGE (<, >, BETWEEN)
 - Không thể tối ưu hóa toán tử ORDER BY

Khi nào không nên dùng Index?

Không nên dùng khi nào

- ❖ Bảng có nhiều thao tác ghi
- ❖ Bảng dữ liệu nhỏ
- ❖ Cột có tính selectivity thấp
- ❖ Giới hạn về lưu trữ
- ❖ Cột chứa giá trị lớn
- ❖ Không nên tham lam đánh index trên nhiều field (2-4)

Cập nhật Bảng thống kê Index

Update ANALYZE TABLE

- ❖ ANALYZE TABLE tên_bảng;
- ❖ Khi nào nên cập nhật thống kê
 - Sau khi thêm/xóa dữ liệu lớn và thay đổi đáng kể (>20%)
 - Sau khi thay đổi cấu trúc bảng như thêm/xóa index hoặc thay đổi schema
 - Định kỳ theo lịch bảo trì (ví dụ: hàng tuần/hàng tháng)
 - Khi nhận thấy kế hoạch truy vấn không tối ưu. Ví dụ khi EXPLAIN cho thấy index không được sử dụng đúng cách